GitHub útmutató a csapattagoknak

Rutherford Scattering Simulation | Ábel, Pityesz és Zsolt | Lépésről lépésre

A cél: mindenki a saját branchén fejleszt, feltölti a változtatásait, majd Pull Requesten keresztül kerülnek a kész részek a közös main branchbe.

Repository: https://github.com/Zsolt421/Rutherford_Scattering_Simulation

1. Egyszeri beállítás – először ezt csináljátok

1. Fogadjátok el a GitHub-meghívást (a repository tulajdonosa a Settings → Collaborators oldalon ad hozzá benneteket).

2. Legyen Git telepítve, és VS Code-ban nyissatok egy terminált. Az alábbi parancsot egy olyan mappában futtassátok, ahová a projektet szeretnétek letölteni:

```
git clone https://github.com/Zsolt421/Rutherford_Scattering_Simulation.git
cd Rutherford_Scattering_Simulation
```

3. Váltsatok a saját branchetekre. Ábel:

```
git fetch origin
git switch Ábel
```

Pityesz:

```
git fetch origin
git switch Pityesz
```

A Zsolt branch ugyanezzel a módszerrel használható: `git switch Zsolt`. Ha a Git nem találja automatikusan a távoli branchet:

```
git switch --track origin/Ábel
```

Pityesznél az utolsó parancsban Ábel helyett Pityesz szerepel. A parancsokban pontosan ugyanazt a kis-/nagybetűt és ékezetet használjátok, mint a GitHubon.

Ellenőrzés: a `git branch --show-current` a saját branch nevét írja ki.

2. Mindennapi munkamenet

Munka megkezdésekor frissítsétek a saját brancheteket:

```
git switch Ábel
git pull --ff-only
```

Pityesz természetesen a `Pityesz` branchre vált. A `git pull --ff-only` megakad, ha eltér a helyi és a távoli előzmény; ilyenkor egyeztessetek, ne erőltessétek a felülírást.

Munka után ellenőrizzétek, mi változott, készítsetek commitot és töltsétek fel:

```
git status
git add .
git commit -m "Implement my module"
```

```
git push
```

Az első feltöltésnél, ha nincs upstream beállítva: `git push -u origin Ábel` (vagy `Pityesz`).

3. Hogyan kerül a kész kód a main branchbe?

GitHubon nyissátok meg a repositoryt → Pull requests → New pull request. Válasszátok: base: main, compare: saját branch. Röviden írjátok le, mit készítettetek el; egy másik csapattag ellenőrizze, és csak utána merge-eljétek.

A main a közös stabil verzió: lehetőleg ne dolgozzatok és ne pusholjatok közvetlenül oda.

4. Ha a main frissült

A saját branchbe a közös frissítéseket így tudjátok átvenni (előbb commitoljátok vagy tegyétek félre a saját módosításokat):

```
git switch Ábel
git fetch origin
git merge origin/main
git push
```

Ha merge-konfliktus van, nyissátok meg VS Code-ban az érintett fájlokat, oldjátok fel a konfliktusokat, majd készítsetek commitot. Pityesz a saját branchnevét használja.

5. Gyakori hibák és megoldásuk

Permission denied / 403: Nincs írási jogosultság vagy a GitHub hitelesítés hiányzik. Ellenőrizzétek a meghívást és a belépett fiókot.

pathspec ... did not match: Futtassátok: git fetch origin; utána váltsatok a pontos branchnevet használva.

nothing to commit: Nem módosult mentett fájl, vagy a módosítás már commitolva van.

rejected / non-fast-forward: A távoli branchen újabb commit van. Előbb git pull --ff-only; elakadás esetén egyeztetés.

not a git repository: Nem a klónozott projekt mappájában vagytok; lépjetek bele a cd paranccsal.

6. Közös szabályok – rövid checklist

☐ Mindig ellenőrzöm, melyik branchen vagyok.

☐ Csak a saját feladatomhoz tartozó fájlokat módosítom; előre egyeztetett interfészeket használunk.

☐ A fordított .exe és ideiglenes fájlok ne kerüljenek a Gitbe (.gitignore).

☐ Commit előtt ellenőrzöm, hogy a program lefordul-e.

☐ A kész modult Pull Requesttel ajánlom fel a mainbe.

Megjegyzés: GitHubon létrehozott branch közös és minden jogosult csapattag számára elérhető. A branch neve nem jelent kizárólagos tulajdont; a fenti munkarend csapatmegállapodás.